The four MPM operators, `default` against `warp`

every line of both, extracted from the source at build time

1. What has a `warp` twin and what does not

operator	`default`	`warp`	per particle per substep, `default` allocates
`mpm_strain`	`mpm_ops.py`	**none** — still `default`	313 B
`mpm_scatter`	`mpm_ops.py`	`mpm_warp.py` `p2g_atomic`	7594 B → **0**
`mpm_grid_update`	`mpm_ops.py`	**none** — still `default`	151 B
`mpm_gather`	`mpm_ops.py`	`mpm_warp.py` `g2p`	6413 B → **0**

Two of the four are **not yet ported**, and they are now the whole remaining frame: with the scatter and the gather fused, `mpm_strain` and `mpm_grid_update` are the only operators still writing intermediates to global memory. The empty column below is a to-do, not a claim that the `default` body is already optimal. Byte figures are from the `TorchDispatchMode` trace in `paper/mpm_warp.pdf` §3.1, at $N = 945\,000$.

The `warp` side of each pair is two pieces: the `@wp.kernel` that runs on the device, and the operator's `forward`, which does nothing but marshal torch tensors into it. They are shown in that order. `wp.from_torch` wraps the SAME device memory rather than copying, so the grid the kernel writes is the field the rest of the substep reads.

2. `mpm_strain` — the deformation gradient update

$\mathbf{F}_p \leftarrow (\mathbf{I} + \Delta t\, \mathbf{C}_p)\mathbf{F}_p$, plus the plastic and liquid branches. A $[N, 3, 3]$ matrix product: it is the operator that fuses most easily and it has not been done.

default `mpm_ops.py:865-917`

```
def forward(self, H, mask=None):
    p = H.level(self.at); dev = p.state.device
    dt = sub_dt(H, self.dt_sub)
    D = p.F.shape[-1]
    eye = torch.eye(D, device=dev).expand(p.n, D, D)
    F = (eye + dt * p.C) @ p.F
    if D == 2:
        a, b, c, d = F[:, 0, 0], F[:, 0, 1], F[:, 1, 0], F[:, 1, 1]
        J = a * d - b * c
    else:
        J = torch.linalg.det(F)
    liquid = getattr(p, "is_liquid", None)
    if _const_any(self, "_c_liquid", liquid): # LIQUID: drop shape memory
        Jc = J.clamp(min=1e-6)
        Jl = torch.sqrt(Jc) if D == 2 else Jc.pow(1.0 / D) # volume-preserving isotropic reset
        F = torch.where(liquid[:, None, None], eye * Jl[:, None, None], F)
    visco = getattr(p, "is_visco", None)
    if _const_any(self, "_c_visco", visco): # VISCOELASTIC (Maxwell): PARTIAL shape reset
        vm = visco # relax F toward isotropic with time-constant tau,
        Fv = F[vm] # keeping VOLUME (J) -> stress builds then decays
        U, sig, Vh = torch.linalg.svd(Fv) # SVD: sig = principal stretches
        Jl = sig.prod(-1).clamp(min=1e-6).pow(1.0 / D) # isotropic (volume-preserving) target stretch
        a = torch.exp(-dt / p.visco_tau[vm].clamp(min=1e-6)) # memory retained this substep: a->1 elastic, a->0 liquid
        sig = Jl[:, None] + (sig - Jl[:, None]) * a[:, None] # pull stretches toward isotropic (shear relaxes, volume kept)
        F = F.clone()
        F[vm] = U @ torch.diag_embed(sig) @ Vh
    snow = getattr(p, "is_snow", None)
    if _const_any(self, "_c_snow", snow): # SNOW: clamp singular values, harden via Jp
        sm = snow; Fs = F[sm]
        if Fs.shape[0] > 0:
            U, sig, Vh = torch.linalg.svd(Fs)
            if D == 3: # proper-rotation sign fix (MPM_3D)
                U = U.clone(); sig = sig.clone(); Vh = Vh.clone()
                negU = torch.det(U) < 0
                U[negU, :, -1] *= -1; sig[negU, -1] *= -1
                negV = torch.det(Vh) < 0
                Vh[negV, -1, :] *= -1; sig[negV, -1] *= -1
            sig_c = sig.clamp(1.0 - 2.5e-2, 1.0 + 7.5e-3)
            F = F.clone(); F[sm] = U @ torch.diag_embed(sig_c) @ Vh
            ratio = sig.prod(-1) / sig_c.prod(-1).clamp(min=1e-6)
            Jp = p.Jp.clone(); Jp[sm] = (Jp[sm] * ratio).clamp(0.6, 20.0)
            p.Jp.copy_(Jp)
    # DORMANT PARTICLES DO NOT DEFORM. `mpm_scatter` masks its weights by occupancy and
    # `mpm_gather` freezes occ==0 rather than advecting it, but this operator integrated F for the
    # reserve regardless -- so a particle waiting to be spawned accumulated an arbitrary deformation
    # for as long as it waited, and was then promoted into real material carrying it. Byte-identical
    # when every particle is live, which is every composition that has no reserve.
    occ = getattr(p, "occ", None)
    if occ is not None:
        live = (occ > 0)[:, None, None]
        F = torch.where(live, F, p.F)
    p.F.copy_(F) # in place; every read of p.F above precedes it
    return {}
```

no `warp` implementation

This operator still runs the `default` body under `warp`. It allocates 313 B per particle per substep — `mul` and `where` on $[N, 3, 3]$, 68 MB each at 945 000 particles.

3. `mpm_scatter` — particle → grid (P2G)

Mass and momentum scattered to the 27 surrounding nodes, with the fixed-corotated stress folded into the affine term. **This is the one with the atomics**: many particles land on the same node, so the writes must be serialised. 108 `wp.atomic_add` per particle (27 nodes × (1 mass + 3 momentum)).

default `mpm_ops.py:279-442`

```
def forward(self, H, mask=None):
    p = H.level(self.at); g = H.field(self.to); dev = p.state.device
    dt = sub_dt(H, self.dt_sub)
    nx, ny, inv_dx, dx = g.nx, g.ny, g.inv_dx, g.dx
    D = p.F.shape[-1]
    periodic = bool(getattr(H, "periodic", False))
    offsets = stencil_offsets(D, dev)
    X, V = p.get("pos"), p.get("vel")
    # external per-cell acceleration from the parent set's accumulated delta (gravity)
    pn = getattr(p, "parent_name", None)
    if pn is not None:
        a_cell = H.delta(pn)
        a_cell = torch.nan_to_num(a_cell, posinf=self.a_max, neginf=-self.a_max).clamp(-self.a_max, self.a_max)
        a_ext = a_cell[p.parent]
    else:
        a_ext = torch.zeros(p.n, D, device=dev)
    part_accel = getattr(H, "part_accel", None)
    if part_accel is not None:
        a_ext = a_ext + part_accel
    # per-particle body force from particle-level force operators (e.g. pulse_to_contraction,
    # drag) -- the symmetric counterpart of the parent-delta path above (gravity).
    a_ext = a_ext + torch.nan_to_num(H.delta(p.name))
    V = V + dt * (a_ext - self.drag * V) # body force + Stokes drag (local; G2P resets V)

    F, C, mass = p.F, p.C, p.mass
    # RESIDUAL STRESS / PRESTRESS (optional, default OFF): compute the fixed-corotated stress
    # relative to a non-identity per-particle REST tensor F_res (multiplicative morphoelastic split
    # F = Fe . F_res, so Fe = F @ F_res_inv is the elastic part). At the mesh rest state F=I this
    # leaves Fe = F_res_inv != I -> a STANDING PRELOAD; an incompatible F_res(x,y) holds a
    # self-equilibrated residual-stress field. Absent buffer -> byte-identical; F_res=I (alpha=0) ->
    # F @ I = F exactly, so the operator truly ablates. Only the STRESS reference shifts; the
    # kinematic F (updated in mpm_strain) is untouched.
    Fres_inv = getattr(p, "F_res_inv", None)
    if Fres_inv is not None:
        F = F @ Fres_inv
    eye = torch.eye(D, device=dev).expand(p.n, D, D)
    if D == 2:
        a, b, c, d = F[:, 0, 0], F[:, 0, 1], F[:, 1, 0], F[:, 1, 1]
        J = a * d - b * c
    else:
        J = torch.linalg.det(F)
    mu, la = p.mu, p.la
    snow = getattr(p, "is_snow", None)
    if _const_any(self, "_c_snow", snow): # snow hardening from the plastic ratio Jp
        h = torch.exp((10.0 * (1.0 - p.Jp)).clamp(-6.0, 6.0))
        mu = torch.where(snow, p.mu * h, p.mu)
        la = torch.where(snow, p.la * h, p.la)
    if D == 2: # analytic 2D polar rotation (bit-identical)
        cs, sn = (F[:, 0, 0] + F[:, 1, 1]), (F[:, 1, 0] - F[:, 0, 1])
        r = torch.sqrt(cs * cs + sn * sn) + 1e-9
        cs, sn = cs / r, sn / r
        R = torch.stack([torch.stack([cs, -sn], -1), torch.stack([sn, cs], -1)], -2)
    elif self.polar == "higham": # Newton polar iteration
        R = _polar_higham(F, self.polar_iters)
    else: # SVD polar rotation R = U Vh (proper rotation)
        # THE SIGN FLIP IS A MULTIPLY, NOT A BOOLEAN MASK, and the two are bit-for-bit the
        # same: `torch.equal` True, max|diff| 0.000e+00 over 1,500 matrices. What changes is
        # the SHAPE of the work. `U[det(U) < 0, :, -1] *= -1` is `aten.nonzero` -- the number
        # of rows it writes depends on how many matrices are improper rotations, which is
        # ~50% here and drifts as the body deforms. Under torch.compile that is a
        # data-dependent shape: dynamo cuts the graph in half at this line and then recompiles
        # on every new count until it hits the recompile limit and falls back to eager for the
        # rest of the run ("___stack2 size mismatch at index 0, expected 13, actual 15").
        # Multiplying the last column by +-1 touches every row unconditionally, so the shape
        # is static and the scatter can be one graph -- which is also the precondition for
        # capturing the substep as a CUDA graph.
        #
        # IN PLACE ON THE CLONE, not `torch.cat` of the columns. Reassembling with `cat`
        # changes the memory layout, the following matmul picks a different kernel, and the
        # result moves by 6e-8 -- small, but enough to break the promotion suite's
        # byte-identity gate for no gain. Writing the column back keeps U contiguous.
        U, sig, Vh = torch.linalg.svd(F)
        U = U.clone(); Vh = Vh.clone()
        sgnU = torch.where(torch.det(U) < 0, -1.0, 1.0)
        sgnV = torch.where(torch.det(Vh) < 0, -1.0, 1.0)
        U[:, :, -1] = U[:, :, -1] * sgnU[:, None]
        Vh[:, -1, :] = Vh[:, -1, :] * sgnV[:, None]
        R = U @ Vh
    stress = 2 * mu[:, None, None] * ((F - R) @ F.transpose(-2, -1)) \
        + eye * (la * J * (J - 1))[:, None, None]
    # optional MLS-MPM ACTIVE STRESS (-A n n^T from pulse_to_active_stress), added to the
    # fixed-corotated elastic stress before the affine scatter. Default off (absent -> None ->
```

```
    # pure elastic); same units / scaling / scatter as the elastic stress. Same H side-channel
    # idiom as part_accel; it feeds the tissue through stress divergence, not a pointwise force.
    act = getattr(H, "active_stress", None)
    if act is not None:
        stress = stress + act
    # TURGOR / OSMOTIC PRESSURE (optional, default OFF): an isotropic OUTWARD pressure carried
    # by this set, written as a per-particle `turgor` buffer by the `mpm_turgor` operator. The
    # sign is the one that makes a cell a cell: tau here is Kirchhoff (positive = tension), a
    # fluid at pressure P has sigma = -P.I, so a POSITIVE `turgor` SUBTRACTS from tau and pushes
    # the material outward -- the same sign the liquid's own `la*J*(J-1)` takes when J < 1
    # (compressed -> pushes out). Absent buffer -> byte-identical; the buffer lives on the LEVEL,
    # so `mpm_turgor at: cytosol` pressurises the cytosol and leaves nucleus/membrane untouched
    # even though all three scatter into the same grid.
    turg = getattr(p, "turgor", None)
    if turg is not None:
        stress = stress - eye * turg[:, None, None]
    if self.store_stress:
        # CAUCHY, NOT KIRCHHOFF. What the lines above build is tau = J.sigma (the fixed-corotated
        # first Piola P times F^T), which is the form MLS-MPM scatters; sigma = tau / J is the
        # stress per unit CURRENT area, which is what "Cauchy stress" means and what a von Mises
        # invariant is normally quoted from. Captured here, after any active stress has been added
        # and BEFORE the dt / p_vol rescale on the next line, so it is the material's stress and
        # not a momentum increment.
        sig = stress / J.abs().clamp_min(1e-9)[:, None, None]
        if getattr(p, "sigma", None) is None or p.sigma.shape != sig.shape:
            p.register_buffer("sigma", torch.zeros_like(sig))
        p.sigma.copy_(sig.detach())
    stress = (-dt * 4 * inv_dx * inv_dx) * p.p_vol[:, None, None] * stress
    affine = stress + mass[:, None, None] * C

    fx, weight, flat = bspline(X, inv_dx, offsets, g.shape, periodic)
    # DORMANT particles (occ==0, e.g. a agent_grow reserve) contribute NOTHING to the grid:
    # mask the scatter weights by occupancy. Byte-identical when all particles are live.
    occ = getattr(p, "occ", None)
    if occ is not None:
        weight = weight * (occ > 0).to(weight.dtype)[:, None]
    dpos_phys = (offsets[None] - fx[:, None, :]) * dx
    mom = mass[:, None, None] * V[:, None, :] + (affine[:, None] @ dpos_phys[..., None]).squeeze(-1)
    # THE GRID IS SHARED BY EVERY SET THAT SCATTERS INTO IT, so only the FIRST scatter of a
    # micro-step may zero it. This used to allocate fresh zeros unconditionally and assign
    # them, which meant a spec with two particle sets kept only the LAST one: nucleus momentum
    # was deposited, then thrown away by the cytosol's scatter, and the grid solve ran on the
    # cytosol alone.
    #
    # WHAT IT LOOKED LIKE, because like the substep-binding defect it does not announce itself.
    # A nucleus falling inside a cytosol shell fell SLOWER than gravity (z = 0.7061 at the tick
    # free-fall puts at 0.6835) and was crushed from an rms thickness of 0.0248 to 0.0093 while
    # still in mid-air, where nothing was touching it. The same nucleus with no cytosol tracked
    # free-fall to four decimals and held 0.0248 exactly. It reads as "the nucleus is too soft"
    # and it is actually "the nucleus is gathering a velocity field it never contributed to":
    # inside the cavity the cytosol deposits no mass, so `gmv / gm.clamp(1e-10)` there is the
    # B-spline tails divided by nearly nothing.
    #
    # WHY IT SURVIVED THIS LONG. Every MPM spec in the corpus scatters ONE set -- multi-material
    # is done with `types:` inside a single `mpm_particle` set (see
    # config/material/material_3d_multimaterial.yaml: jelly + water + snow, one set, one
    # scatter). Multi-SET MPM is what the composed cell introduced, and it is the only thing
    # this ever affected. With one set the stamp is always stale and the zeros are always
    # fresh, so single-set runs stay bit-identical.
    # STEP 3: WHO ZEROES THE GRID IS STATIC, so it is not asked at run time. This used to read
    # `H.micro`, a python int the engine advances every substep, and compare it to a stamp on
    # the field. That works, but it is a per-substep side effect no CUDA-graph replay can
    # reproduce -- and it made dynamo recompile on every substep, because it specialises on
    # integer attributes of an nn.Module. The engine binds the i-th OCCURRENCE of a token to
    # the i-th INSTANCE, so occurrence 0 of `mpm_scatter` for a given grid is ALWAYS the first
    # one in a substep: the engine stamps `_zeroes_grid` on it when `inst` is built.
    _fresh = getattr(self, "_zeroes_grid", True)
    # STEP 2: WRITE INTO THE FIELD'S OWN BUFFERS, never a fresh allocation. Assigning
    # `g.m = torch.zeros(...)` rebinds the attribute to new storage every substep; a captured
    # graph holds the address it saw at capture time, so the replay silently writes somewhere
    # the rest of the run no longer reads. Measured: capture SUCCEEDS with this left as it was,
    # raises nothing, and produces a 6-tick checksum of 22816.79 against the correct 22132.22.
    gm, gmv, gc = g.m, g.mv, g.c
    if _fresh:
        gm.zero_(); gmv.zero_(); gc.zero_()
    gm.index_add_(0, flat, (weight * mass[:, None]).reshape(-1))
    gmv.index_add_(0, flat, (weight[..., None] * mom).reshape(-1, D))
    liquid = getattr(p, "is_liquid", None)
    if _const_any(self, "_c_liquid", liquid): # liquid colour for the CSF surface tension
        lw = (weight * (mass * liquid.to(mass.dtype))[:, None]).reshape(-1)
        gc.index_add_(0, flat, lw)
    return {}
```

warp — the device kernel mpm_warp.py:60-124

```
@wp.kernel
def p2g_atomic(X: wp.array(dtype=wp.vec3), V: wp.array(dtype=wp.vec3),
               C: wp.array(dtype=wp.mat33), F: wp.array(dtype=wp.mat33),
               MASS: wp.array(dtype=float), MU: wp.array(dtype=float),
               LA: wp.array(dtype=float), PVOL: wp.array(dtype=float),
               AEXT: wp.array(dtype=wp.vec3),
```

```
          GM: wp.array(dtype=float), GMV: wp.array(dtype=wp.vec3),
          ng: int, dx: float, dt: float, drag: float, iters: int):
p = wp.tid()
inv_dx = 1.0 / dx

x = X[p]
v = V[p] + dt * (AEXT[p] - drag * V[p]) # body force + Stokes drag, as the torch op does
mass = MASS[p]
Fp = F[p]
Cp = C[p]

J = wp.determinant(Fp)
R = polar_R(Fp, iters)
# fixed-corotated Kirchhoff stress: 2 mu (F - R) F^T + I la J (J - 1)
S = 2.0 * MU[p] * ((Fp - R) * wp.transpose(Fp)) + wp.identity(n=3, dtype=float) * (
    LA[p] * J * (J - 1.0))
S = S * ((-dt * 4.0 * inv_dx * inv_dx) * PVOL[p])
affine = S + Cp * mass

base = wp.vec3(wp.floor(x[0] * inv_dx - 0.5),
               wp.floor(x[1] * inv_dx - 0.5),
               wp.floor(x[2] * inv_dx - 0.5))
fx = wp.vec3(x[0] * inv_dx - base[0], x[1] * inv_dx - base[1], x[2] * inv_dx - base[2])

for i in range(3):
    wi = float(0.0)
    if i == 0:
        wi = 0.5 * (1.5 - fx[0]) * (1.5 - fx[0])
    elif i == 1:
        wi = 0.75 - (fx[0] - 1.0) * (fx[0] - 1.0)
    else:
        wi = 0.5 * (fx[0] - 0.5) * (fx[0] - 0.5)
    for j in range(3):
        wj = float(0.0)
        if j == 0:
            wj = 0.5 * (1.5 - fx[1]) * (1.5 - fx[1])
        elif j == 1:
            wj = 0.75 - (fx[1] - 1.0) * (fx[1] - 1.0)
        else:
            wj = 0.5 * (fx[1] - 0.5) * (fx[1] - 0.5)
        for k in range(3):
            wk = float(0.0)
            if k == 0:
                wk = 0.5 * (1.5 - fx[2]) * (1.5 - fx[2])
            elif k == 1:
                wk = 0.75 - (fx[2] - 1.0) * (fx[2] - 1.0)
            else:
                wk = 0.5 * (fx[2] - 0.5) * (fx[2] - 0.5)
            w = wi * wj * wk
            gi = wp.clamp(int(base[0]) + i, 0, ng - 1)
            gj = wp.clamp(int(base[1]) + j, 0, ng - 1)
            gk = wp.clamp(int(base[2]) + k, 0, ng - 1)
            idx = (gi * ng + gj) * ng + gk
            dpos = wp.vec3((float(i) - fx[0]) * dx,
                           (float(j) - fx[1]) * dx,
                           (float(k) - fx[2]) * dx)
            mom = mass * v + affine * dpos
            wp.atomic_add(GM, idx, w * mass)
            wp.atomic_add(GMV, idx, w * mom)
```

warp — the launch `mpm_warp.py:136-191`

```
def forward(self, H, mask=None):
    if not HAVE_WARP:
        raise RuntimeError("mpm_scatter[warp] needs warp-lang; none importable")
    from plexus.operators.mpm_ops import sub_dt
    p = H.level(self.at); g = H.field(self.to); dev = p.state.device
    D = p.F.shape[-1]
    if D != 3 or str(dev) == "cpu":
        raise RuntimeError(f"mpm_scatter[warp] is 3D CUDA only (got dim={D}, dev={dev})")
    dt = sub_dt(H, self.dt_sub)

    X, V = p.get("pos").contiguous(), p.get("vel").contiguous()
    pn = getattr(p, "parent_name", None)
    if pn is not None:
        ac = torch.nan_to_num(H.delta(pn), posinf=self.a_max, neginf=-self.a_max
                              ).clamp(-self.a_max, self.a_max)
        a_ext = ac[p.parent]
    else:
        a_ext = torch.zeros(p.n, D, device=dev)
    pa = getattr(H, "part_accel", None)
    if pa is not None:
        a_ext = a_ext + pa
    a_ext = (a_ext + torch.nan_to_num(H.delta(p.name))).contiguous()

    gm, gmv = g.m, g.mv
    if getattr(self, "_zeroes_grid", True):
        gm.zero_(); gmv.zero_(); g.c.zero_()

    # ZERO-COPY. `wp.from_torch` wraps the same device memory, so the grid the kernel writes IS
    # the field the rest of the substep reads -- no staging, and the in-place discipline the
```

```
        # capture guard depends on is preserved.
        wdev = f"cuda:{dev.index or 0}"
        n = int(p.n)
        wp.launch(
            p2g_atomic, dim=n, device=wdev,
            inputs=[wp.from_torch(X, dtype=wp.vec3), wp.from_torch(V, dtype=wp.vec3),
                    wp.from_torch(p.C.contiguous(), dtype=wp.mat33),
                    wp.from_torch(p.F.contiguous(), dtype=wp.mat33),
                    wp.from_torch(p.mass.contiguous()), wp.from_torch(p.mu.contiguous()),
                    wp.from_torch(p.la.contiguous()), wp.from_torch(p.p_vol.contiguous()),
                    wp.from_torch(a_ext, dtype=wp.vec3),
                    wp.from_torch(gm), wp.from_torch(gmv.view(-1, 3), dtype=wp.vec3),
                    int(g.nx), float(g.dx), float(dt), float(self.drag), int(self.polar_iters)])
        return {}


# ============================================================================================================
# G2P -- `mpm_gather[implementation: warp]`
#
# THE SCATTER WAS 64% OF THE FRAME AND IS NOW ~21%; PROFILED AFTER THAT, THE GATHER IS 60.5%.
# It is also by far the easier kernel: grid -> particle is a pure READ. Each particle reads the
# velocity of its 27 neighbouring nodes and forms two weighted sums (the new velocity, and the
# affine matrix C). Nothing is shared, nothing collides, there are no atomics and no sort. The
# PyTorch version is slow for one reason only: it materialises [N, 27, 3] and [N, 27, 3, 3]
# intermediates through global memory, and never needs to.
# ============================================================================================================
if HAVE_WARP:
```

4. `mpm_grid_update` — the grid solve

$\mathbf{v}_i = (m\mathbf{v})_i / \max(m_i, \epsilon)$, then every boundary condition the model owns: gravity, walls, moving plates, CSF surface tension, buoyancy. The longest of the four and the least like the others — it is $O(\text{cells})$, not $O(\text{particles})$, so a port is a different kernel shape.

default `mpm_ops.py:627-764`

```
def forward(self, H, mask=None):
    g = H.field(self.at); dev = g.m.device
    dt = sub_dt(H, self.dt_sub)
    nx, ny, inv_dx, dx = g.nx, g.ny, g.inv_dx, g.dx
    D = g.dim
    periodic = bool(getattr(H, "periodic", False))
    wd = self.wall_damp
    gm, gmv, gc = g.m, g.mv, g.c
    gv = gmv / gm.clamp(min=1e-10)[:, None]

    if D == 2: # --- 2D: verbatim (bit-identical) ---
        surf = self.surface_tension
        # CACHED, not dropped. Unlike the material masks this one is not run-constant in
        # PRINCIPLE -- gc is rebuilt every substep -- but it is settled by the first grid
        # solve, which always runs after every scatter of the substep. Deleting the guard
        # instead would turn gv into gv + 0.0 on a no-liquid spec and flip -0.0 to +0.0.
        if getattr(self, "_c_csf", None) is None:
            self._c_csf = bool(surf > 0.0 and bool((gc > 0).any()))
        if self._c_csf: # CSF continuum surface force
            c = gc.view(nx, ny)
            cx = (torch.roll(c, -1, 0) - torch.roll(c, 1, 0)) * (0.5 * inv_dx)
            cy = (torch.roll(c, -1, 1) - torch.roll(c, 1, 1)) * (0.5 * inv_dx)
            gmag = torch.sqrt(cx * cx + cy * cy); eps = 1e-6
            nxg, nyg = cx / (gmag + eps), cy / (gmag + eps)
            kappa = -((torch.roll(nxg, -1, 0) - torch.roll(nxg, 1, 0)) * (0.5 * inv_dx)
                      + (torch.roll(nyg, -1, 1) - torch.roll(nyg, 1, 1)) * (0.5 * inv_dx))
            fmask = (gmag > 0.02 * gmag.max()).to(c.dtype)
            stfx = (surf * kappa * cx * fmask).view(-1); stfy = (surf * kappa * cy * fmask).view(-1)
            inv_m = (dx * dx) / gm.clamp(min=1e-8)
            gv = gv + dt * torch.stack([stfx * inv_m, stfy * inv_m], dim=1)

        if not periodic: # reflective domain walls
            gv = gv.view(nx, ny, 2)
            ix = torch.arange(nx, device=dev); iy = torch.arange(ny, device=dev); bnd = 3
            lox, hix = ix < bnd, ix > nx - bnd
            loy, hiy = iy < bnd, iy > ny - bnd
            gv[lox, :, 0] = gv[lox, :, 0].clamp(min=0); gv[hix, :, 0] = gv[hix, :, 0].clamp(max=0)
            gv[:, loy, 1] = gv[:, loy, 1].clamp(min=0); gv[:, hiy, 1] = gv[:, hiy, 1].clamp(max=0)
            if wd != 1.0:
                gl = gv[lox, :, 1]; gv[lox, :, 1] = torch.where(gl > 0, gl * wd, gl)
                gh = gv[hix, :, 1]; gv[hix, :, 1] = torch.where(gh > 0, gh * wd, gh)
                gv[:, loy, 0] = gv[:, loy, 0] * wd
                gv[:, hiy, 0] = gv[:, hiy, 0] * wd
            gv = gv.view(nx * ny, 2)
        walls = self._walls(H, g, dev)
        if wd != 1.0 and _const_any(self, "_c_walls2d", walls): # friction in cells touching obstacles
            w2 = walls.view(nx, ny)
            near = (torch.roll(w2, 1, 0) | torch.roll(w2, -1, 0)
                    | torch.roll(w2, 1, 1) | torch.roll(w2, -1, 1)) & ~w2
            gvv = gv.view(nx, ny, 2); gx_ = gvv[..., 0]; gy_ = gvv[..., 1]
            gvv[..., 0] = torch.where(near, gx_ * wd, gx_)
            gvv[..., 1] = torch.where(near & (gy_ > 0), gy_ * wd, gy_)
            gv = gvv.view(nx * ny, 2)
        gv = torch.where(walls[:, None], torch.zeros_like(gv), gv) # interior wall BC
    else: # --- 3D: CSF + reflective box walls ---
        # SURFACE TENSION IN 3D. Until now this whole term lived inside the `D == 2` branch
        # above, so `surface_tension: 60.0` in a `dim: 3` spec was read, stored, and never
        # used. It cost a rung of the cell ladder: a liquid cytosol dropped onto the floor
        # spread into a one-particle-thick puddle covering the entire domain, which reads as
        # "the liquid is too soft" and is actually "there is no cohesion term at all". A liquid
        # in MLS-MPM has mu = 0 by construction -- it resists volume change and nothing else --
        # so surface tension is not a refinement on top of the constitutive model, it is the
        # ONLY thing that makes a droplet a droplet.
        #
        # SAME CSF AS 2D, WRITTEN OVER D AXES. Brackbill's continuum surface force: the liquid
        # colour `gc` deposited by the scatter is smooth across the interface, its gradient is
        # the interface normal times the interface's sharpness, and the divergence of the unit
        # normal is the curvature. f = sigma * kappa * grad(c) then pulls a bulge in and pushes
        # a dimple out. The 2D code above is left untouched rather than generalised, because it
        # is on the promotion path and must stay bit-identical.
        surf = self.surface_tension
        if getattr(self, "_c_csf", None) is None:
            self._c_csf = bool(surf > 0.0 and bool((gc > 0).any()))
        if self._c_csf:
            c = gc.view(*g.shape)
            grad = [(torch.roll(c, -1, k) - torch.roll(c, 1, k)) * (0.5 * inv_dx)
                    for k in range(D)]
            gmag = torch.sqrt(sum(gk * gk for gk in grad))
            eps = 1e-6
            nrm = [gk / (gmag + eps) for gk in grad]
            kappa = -sum((torch.roll(nrm[k], -1, k) - torch.roll(nrm[k], 1, k)) * (0.5 * inv_dx)
                         for k in range(D))
```

```
        # ONLY WHERE THERE IS AN INTERFACE. In the bulk `grad(c)` is numerical noise and
        # the unit normal built from it is a random direction, so an unmasked curvature
        # injects a random force into every interior node. The 2% of the peak gradient is
        # the 2D threshold, kept.
        fmask = (gmag > 0.02 * gmag.max()).to(c.dtype)
        inv_m = (dx ** D) / gm.clamp(min=1e-8) # force -> acceleration on the node
        gv = gv + dt * torch.stack(
            [(surf * kappa * grad[k] * fmask).view(-1) * inv_m for k in range(D)], dim=1)
    if not periodic:
        shape = g.shape; bnd = 3
        gv = gv.view(*shape, D)
        for k in range(D):
            n_k = shape[k]
            idx = torch.arange(n_k, device=dev)
            shp = [1] * D; shp[k] = n_k
            lo_m = (idx < bnd).view(shp); hi_m = (idx > n_k - bnd).view(shp)
            ck = gv[..., k]
            ck = torch.where(lo_m, ck.clamp(min=0), ck) # don't penetrate the wall
            ck = torch.where(hi_m, ck.clamp(max=0), ck)
            gv[..., k] = ck
            if wd != 1.0: # tangential friction on the wall slabs
                slab = lo_m | hi_m
                for j in range(D):
                    if j == k:
                        continue
                    cj = gv[..., j]
                    gv[..., j] = torch.where(slab, cj * wd, cj)
        gv = gv.view(g.n_cells, D)
    if self.plate_axis is not None and self.plate_gap_half > 0.0:
        gv = self._plate_bc(H, g, gv, dev, dt)
    walls = self._walls3d(H, g, dev) # solid 3D obstacles (box / sphere)
    if _const_any(self, "_c_walls3d", walls):
        gv = torch.where(walls[:, None], torch.zeros_like(gv), gv) # no-slip: zero grid velocity inside
# BUOYANCY IS NOT A DIMENSION BRANCH, and putting it here made it look like one. Inserted
# in 40e1d0c9 between `if D == 2:` and its `else:`, it STOLE THAT ELSE: the 3D wall code
# then ran whenever buoyancy was zero -- for a 2D spec too, where `_walls3d` unpacks
# `nx, ny, nz = g.shape` and dies -- and was SKIPPED whenever buoyancy was on, so every 3D
# buoyant run had no wall boundary condition at all. Both halves of that were invisible
# until a 2D spec was written without buoyancy. It now sits after the branch, in both
# dimensions, which is what a body force on the grid is.
if self.buoyancy != 0.0:
    # rho at each node from the mass the particles deposited there. `gm` is a node mass and
    # dx^D its volume, so this is a genuine density and the comparison with `rho_ref` is
    # dimensionally honest rather than a tuned ratio.
    _rho = gm / (dx ** D)
    _act = _rho > 1e-9 # empty nodes have no buoyancy
    _f = torch.zeros_like(_rho)
    _f[_act] = (_rho[_act] - self.rho_ref) / _rho[_act]
    _dir = torch.zeros(D, device=dev, dtype=gv.dtype)
    if self.buoy_dir is not None:
        _dir[:len(self.buoy_dir)] = torch.as_tensor(self.buoy_dir, device=dev, dtype=gv.dtype)
    else:
        _dir[1 if D == 2 else 2] = -1.0 # "down" is -y in 2D, -z in 3D
    gv = gv + dt * self.buoyancy * _f[:, None] * _dir[None, :]
g.v.copy_(gv) # in place: a captured graph holds this address
return {}
```

no `warp` implementation

This operator still runs the `default` body under `warp`. It allocates 151 B per particle per substep. Note that CUDA-graph capture already removes most of what this costs, by holding its intermediates in a resident private pool instead of re-allocating them 7 times a frame (`mpm_warp.pdf` §5.3).

5. `mpm_gather` — grid → particle (G2P)

Velocity and the affine matrix **C** read back from the same 27 nodes, then the advection. **A pure read**: no two threads write the same place, so there is nothing to coordinate — no atomics, no sort, no ownership question. That is why it was ported first.

default `mpm_ops.py:790-845`

```
def forward(self, H, mask=None):
    p = H.level(self.at); g = H.field(self.frm); dev = p.state.device
    dt = sub_dt(H, self.dt_sub)
    inv_dx, dx = g.inv_dx, g.dx
    D = p.F.shape[-1]
    periodic = bool(getattr(H, "periodic", False))
    # CACHED, because it is constant for the whole run and `float()` on a tensor element is a
    # host sync. Read fresh every call it cost a GPU->CPU round trip per substep, and under
    # torch.compile it broke the graph in the middle of the gather on every single call.
    # Computed once, the branch is False on every later call and never enters the traced graph.
    if getattr(self, "_box", None) is None:
        self._box = [float(b) for b in
                     getattr(H, "world_size", torch.tensor([g.width, 1.0]))][:D]
    box = self._box
    offsets = stencil_offsets(D, dev); S = offsets.shape[0]
    X, V = p.get("pos"), p.get("vel")
    fx, weight, flat = bspline(X, inv_dx, offsets, g.shape, periodic)
    gvn = g.v[flat].view(p.n, S, D)
    new_V = (weight[..., None] * gvn).sum(1)
    dpos_grid = offsets[None] - fx[:, None, :]
    new_C = 4 * inv_dx * (weight[..., None, None] * (gvn[..., :, None] @ dpos_grid[..., None, :])).sum(1)
    new_V = torch.nan_to_num(new_V)
    if self.wall_damp != 1.0 and not periodic: # inelastic wall contact (solids)
        cb = self.wall_contact
        near = torch.zeros(p.n, dtype=torch.bool, device=dev)
        for k in range(D):
            near = near | (X[:, k] < cb) | (X[:, k] > box[k] - cb)
        liquid = getattr(p, "is_liquid", None)
        if liquid is not None:
            near = near & ~liquid
        new_V = torch.where(near[:, None], new_V * self.wall_damp, new_V)
    sp = new_V.norm(dim=1, keepdim=True).clamp(min=1e-9)
    vmax = min(self.vmax, 0.4 * dx / dt) # CFL velocity cap
    new_V = new_V * (sp.clamp(max=vmax) / sp)
    new_C = torch.nan_to_num(new_C)
    Xn = torch.nan_to_num(X + dt * new_V, nan=0.5)
    if periodic:
        Xn = torch.stack([torch.remainder(Xn[:, k], box[k]) for k in range(D)], dim=1)
    else:
        Xn = torch.stack([Xn[:, k].clamp(2 * dx, box[k] - 2 * dx) for k in range(D)], dim=1)
    # DORMANT particles (occ==0, a agent_grow reserve) are FROZEN -- not advected -- so they sit as a
    # compact reservoir until agent_grow activates + repositions them. Byte-identical when all are live.
    occ = getattr(p, "occ", None)
    if occ is not None:
        live = occ > 0
        Xn = torch.where(live[:, None], Xn, X)
        new_V = torch.where(live[:, None], new_V, V)
        new_C = torch.where(live[:, None, None], new_C, p.C)
    # IN PLACE. Every read of X / V above happens before this write, and this operator declares
    # MAY_MUTATE_INTEGRATED_STATE, so the engine's tick-0 integration-invariant guard does not
    # apply to it. The clone-and-rebind it replaces gave `p.state` a new address every substep.
    pa, pb = p.state_schema["pos"]; va, vb = p.state_schema["vel"]
    p.state[:, pa:pb] = Xn
    p.state[:, va:vb] = new_V
    p.C.copy_(new_C)
    return {}
```

warp — the device kernel `mpm_warp.py:193-268`

```
@wp.kernel
def g2p(X: wp.array(dtype=wp.vec3), STATE: wp.array2d(dtype=float),
        C: wp.array(dtype=wp.mat33), GV: wp.array(dtype=wp.vec3),
        OCC: wp.array(dtype=float), LIQ: wp.array(dtype=float),
        pa: int, va: int, ngx: int, ngy: int, ngz: int, dx: float, dt: float,
        wall_damp: float, wall_contact: float, vmax: float,
        bx: float, by: float, bz: float, has_liq: int):
    p = wp.tid()
    inv_dx = 1.0 / dx
    x = X[p]
    base = wp.vec3(wp.floor(x[0] * inv_dx - 0.5),
                   wp.floor(x[1] * inv_dx - 0.5),
                   wp.floor(x[2] * inv_dx - 0.5))
    fx = wp.vec3(x[0] * inv_dx - base[0], x[1] * inv_dx - base[1], x[2] * inv_dx - base[2])

    newv = wp.vec3(0.0, 0.0, 0.0)
    newC = wp.mat33(0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0)
    for i in range(3):
        wi = float(0.0)
        if i == 0:
```

```
            wi = 0.5 * (1.5 - fx[0]) * (1.5 - fx[0])
        elif i == 1:
            wi = 0.75 - (fx[0] - 1.0) * (fx[0] - 1.0)
        else:
            wi = 0.5 * (fx[0] - 0.5) * (fx[0] - 0.5)
        for j in range(3):
            wj = float(0.0)
            if j == 0:
                wj = 0.5 * (1.5 - fx[1]) * (1.5 - fx[1])
            elif j == 1:
                wj = 0.75 - (fx[1] - 1.0) * (fx[1] - 1.0)
            else:
                wj = 0.5 * (fx[1] - 0.5) * (fx[1] - 0.5)
            for k in range(3):
                wk = float(0.0)
                if k == 0:
                    wk = 0.5 * (1.5 - fx[2]) * (1.5 - fx[2])
                elif k == 1:
                    wk = 0.75 - (fx[2] - 1.0) * (fx[2] - 1.0)
                else:
                    wk = 0.5 * (fx[2] - 0.5) * (fx[2] - 0.5)
                w = wi * wj * wk
                # row-major over `g.shape`, EXACTLY as `bspline` flattens it: axis 0 spans the
                # world width and carries `nx`, axes 1-2 span [0,1] and carry `ny`.
                gi = wp.clamp(int(base[0]) + i, 0, ngx - 1)
                gj = wp.clamp(int(base[1]) + j, 0, ngy - 1)
                gk = wp.clamp(int(base[2]) + k, 0, ngz - 1)
                gv = GV[(gi * ngy + gj) * ngz + gk]
                dpos = wp.vec3(float(i) - fx[0], float(j) - fx[1], float(k) - fx[2])
                newv = newv + w * gv
                newC = newC + (4.0 * inv_dx * w) * wp.outer(gv, dpos)

    # inelastic wall contact for SOLIDS: a liquid is handled by the asymmetric grid wall
    # friction instead, so pinning it here would stop it draining.
    if wall_damp != 1.0:
        near = (x[0] < wall_contact or x[0] > bx - wall_contact or
                x[1] < wall_contact or x[1] > by - wall_contact or
                x[2] < wall_contact or x[2] > bz - wall_contact)
        if has_liq == 1 and LIQ[p] > 0.0:
            near = False
        if near:
            newv = newv * wall_damp

    sp = wp.length(newv)
    if sp > vmax:
        newv = newv * (vmax / sp)
    xn = x + dt * newv
    xn = wp.vec3(wp.clamp(xn[0], 2.0 * dx, bx - 2.0 * dx),
                 wp.clamp(xn[1], 2.0 * dx, by - 2.0 * dx),
                 wp.clamp(xn[2], 2.0 * dx, bz - 2.0 * dx))

    if OCC[p] <= 0.0: # DORMANT particles are frozen, not advected
        return
    STATE[p, pa + 0] = xn[0]; STATE[p, pa + 1] = xn[1]; STATE[p, pa + 2] = xn[2]
    STATE[p, va + 0] = newv[0]; STATE[p, va + 1] = newv[1]; STATE[p, va + 2] = newv[2]
    C[p] = newC
```

warp — the launch mpm_warp.py:279-312

```
def forward(self, H, mask=None):
    if not HAVE_WARP:
        raise RuntimeError("mpm_gather[warp] needs warp-lang")
    from plexus.operators.mpm_ops import sub_dt
    p = H.level(self.at); g = H.field(self.frm); dev = p.state.device
    D = p.F.shape[-1]
    if D != 3 or str(dev) == "cpu" or bool(getattr(H, "periodic", False)):
        raise RuntimeError("mpm_gather[warp] is 3D, non-periodic, CUDA only")
    dt = sub_dt(H, self.dt_sub)
    if getattr(self, "_box", None) is None:
        self._box = [float(b) for b in
                     getattr(H, "world_size", torch.tensor([g.width, 1.0]))][:D]
    bx, by, bz = self._box
    pa, _pb = p.state_schema["pos"]; va, _vb = p.state_schema["vel"]
    occ = getattr(p, "occ", None)
    if occ is None:
        occ = torch.ones(p.n, device=dev)
    liq = getattr(p, "is_liquid", None)
    has_liq = 1 if liq is not None else 0
    liqf = (liq.float().contiguous() if liq is not None
            else torch.zeros(p.n, device=dev))
    vmax = min(self.vmax, 0.4 * float(g.dx) / float(dt))
    wdev = f"cuda:{dev.index or 0}"
    wp.launch(g2p, dim=int(p.n), device=wdev,
              inputs=[wp.from_torch(p.get("pos").contiguous(), dtype=wp.vec3),
                      wp.from_torch(p.state),
                      wp.from_torch(p.C, dtype=wp.mat33),
                      wp.from_torch(g.v.view(-1, 3).contiguous(), dtype=wp.vec3),
                      wp.from_torch(occ.contiguous()), wp.from_torch(liqf),
                      int(pa), int(va), int(g.shape[0]), int(g.shape[1]), int(g.shape[2]),
                      float(g.dx), float(dt),
                      float(self.wall_damp), float(self.wall_contact), float(vmax),
```

```
                    float(bx), float(by), float(bz), int(has_liq)])
    return {}
```